

Your systems, remembered.

Akashic turns scattered repository context into a local, Git-backed knowledge layer that stays inspectable, portable, and yours.



- › akashic init
- › akashic attach ../billing
- › akashic generate
- › akashic serve

01 / THE FRACTURE

The code is connected. The knowledge is not.

Every handoff begins with archaeology across people, wikis, repositories, and stale diagrams.

TRIBAL KNOWLEDGE

Lives in heads and disappears at the worst moment.

STALE WIKIS

Drift quietly until nobody trusts the page.

REPO SILOS

Explain a service, never the system around it.

LOCKED INDEXES

Useful context, trapped behind someone else's backend.

Result **Every new engineer and every coding session starts from zero.**

One repository becomes the memory of the system.

Plain Markdown. Local execution. Git history. A familiar browser. No knowledge backend required.

01

LOCAL

Runs entirely on your machine.

02

PORTABLE

Moves as ordinary Git.

03

LEGIBLE

Markdown for people and tools.

04

AUDITABLE

Every change is reviewable.

The product is not another source of truth.

It gives your existing source of truth a durable memory.

03 / THE LOOP

Four commands. One living knowledge base.

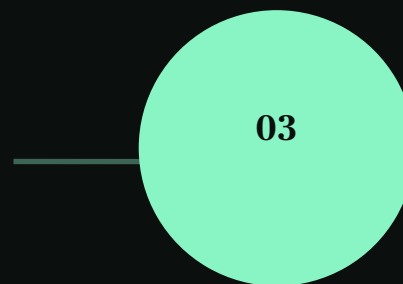
Akashic stays thin: scaffold, register, compose, validate, and serve.

**INIT**

Create the knowledge repo

**ATTACH**

Register source repos in place

**GENERATE**

Launch an interactive authoring session

**SERVE**

Browse, search, edit, and share

USER APPROVES LIVE

The session remains visible and interactive. Git stays under the user's control.

04 / PRODUCT BOUNDARY

A conductor, not another orchestra.

Akashic coordinates the tools developers already use and keeps the system intentionally replaceable.



Portable by design. Personal where necessary.

Shared intent is committed. Machine-specific paths stay local and ignored.

COMMITTED

.akashic/config.yaml

- repository names
- provider choice
- site port
- generation hints

GITIGNORED

.akashic/config.local.yaml

- absolute paths
- per-machine mapping
- safe teammate clones
- no path leakage

Same knowledge.

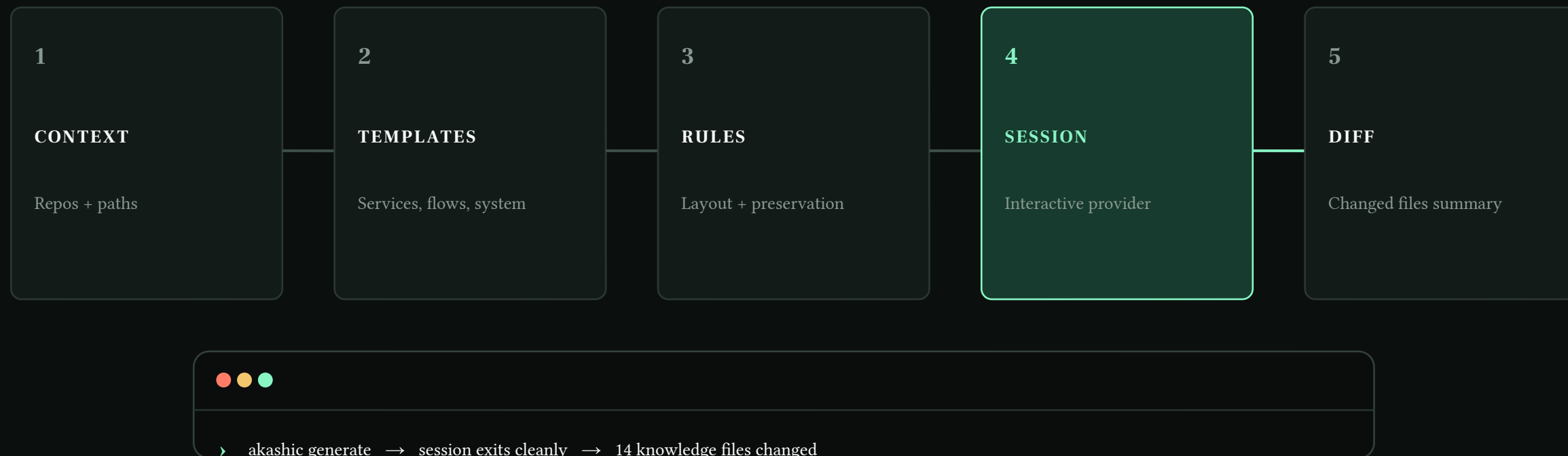
Different machines.

Zero broken paths.

06 / GENERATION

The coding session becomes the documentation engine.

Akashic composes one complete brief, then lets the configured provider read, reason, and write in full view.



07 / DOCUMENT CONTRACT

Generated structure. Protected human judgment.

Each file declares where it came from and reserves a durable space for manual context.

service-payments.md

```
generated:      true
type:           service
name:           payments
sources:        [/repos/payments]
updated_at:     2026-07-14T...
generator:      akashic@0.1.0
```

GENERATED BODY

Refreshable system facts, diagrams, interfaces, and flows.

HUMAN START ↔ HUMAN END

Protected notes, decisions, caveats, and team memory.

The product earns trust by leaving control visible.

Akashic makes safety a workflow property, not a promise hidden in a backend.

SOURCE CODE	Registered in place	Never moved or rewritten by Akashic
GENERATION	Interactive TTY	Permissions approved live
GIT	No staging or commits	The user owns every diff
KNOWLEDGE	Plain files	Inspectable with any editor

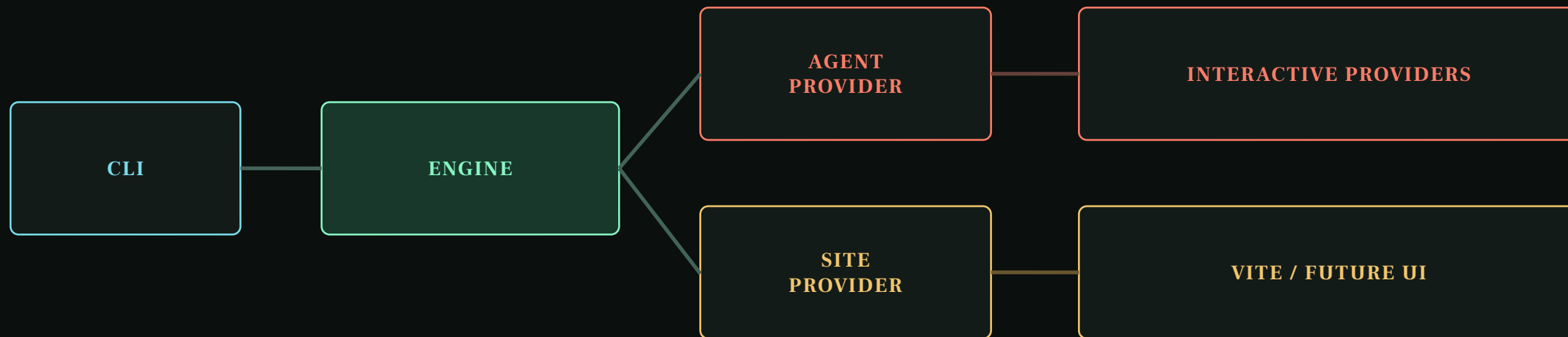
KNOWN LIMITATION

Source-repo write protection is prompt-enforced in v1. Generate against clean, committed trees.

09 / REPLACEABLE SEAMS

Built for today. Deliberately swappable tomorrow.

Interfaces isolate the places most likely to evolve without forcing a rewrite of the CLI core.



Thin entrypoints

Stable domain core

Replace behind contracts

Evolve independently

Knowledge should feel navigable, not merely stored.

The local site turns structured Markdown into a fast visual layer for discovery, reading, and editing.

The screenshot displays a web application interface for 'Payments Service'. The top bar shows 'localhost:4173'. On the left, a sidebar contains a search bar with 'payments' and a navigation menu with categories: SYSTEM, SERVICES, FLOWS, ENTITIES, and ADRS. The 'SERVICES' category is selected, and 'payments' is highlighted. The main content area features the title 'Payments Service' with a 'GENERATED' badge. Below the title is a description: 'Owns payment intent creation, confirmation, and ledger handoff.' At the bottom, three colored boxes represent different components: '12 endpoints' (dark green), '04 flows' (dark blue), and '03 ADRs' (dark brown).

localhost:4173

SEARCH KNOWLEDGE

payments

SYSTEM

SERVICES

payments

identity

FLOWS

ENTITIES

ADRS

Payments Service

GENERATED

Owns payment intent creation, confirmation, and ledger handoff.

12 endpoints

04 flows

03 ADRs

Sharp scope is a feature.

The first release proves the knowledge loop without pretending to solve every adjacent problem.

IN

- **Init, attach, detach, list**
- **Interactive generation**
- Two provider adapters
- Structured docs + human sections
- Local site + static build
- Doctor and status

NOT YET

- Cloud backend or auth
- Headless scheduling
- Deterministic orchestration
- Automatic merge engine
- In-browser Git workflows
- Knowledge graph or vector DB

Prove the loop: **source → understanding → Git → browser → better work**

12 / VALIDATION

Test the behavior users can actually observe.

The provider boundary keeps the slow, unpredictable session out of deterministic product tests.

CLI Init, attach, generate, doctor, status

PROVIDER Availability, version, invocation

PROMPT Composition and overrides

CONFIG Merge and validation

GIT Observable repository state

SITE Static build contract

FAKE PROVIDER

Records context

Writes fixtures

Keeps CI fast

The risky edges are explicit, not buried.

Each accepted limitation has a visible mitigation and a clear path forward.

WRITE SCOPE	Provider can technically reach source repos	Clean trees + live approval	Future sandbox scoping
INFERRED DOCS	ADRs and glossary may be weakly grounded	Review before commit	Grounded/inferred labels
PARALLELISM	One session owns fan-out in v1	Prompt-level guidance	Deterministic runner
PORTABILITY	Local paths differ by machine	Split configuration	Already solved in MVP
RISK	MITIGATION		NEXT HARDENING

The MVP succeeds when context survives the handoff.

A working loop is more valuable than a long feature list.

01 INIT

A knowledge repository exists

02 ATTACH

Source repos stay exactly where they are

03 GENERATE

Structured docs appear through a live session

04 PRESERVE

Human notes survive regeneration

05 BROWSE

The system is searchable locally

06 HISTORY

Every change remains ordinary Git

The outcome is not “documentation generated.”

The outcome is “understanding retained.”

THE THESIS

Systems forget.
Git remembers.
Akashic connects the two.



```
› akashic init  
  
› akashic attach ../system  
  
› akashic generate  
  
› git diff  
  
› akashic serve
```

LOCAL

GIT-BACKED

HUMAN-CONTROLLED

akashic

Your systems, remembered.